

电子设计基础训练

电子信息工程学院
空天电子信息国家级实验教学示范中心
空天电子信息国家级虚拟仿真实验教学中心





Arduino神经网络语音识别

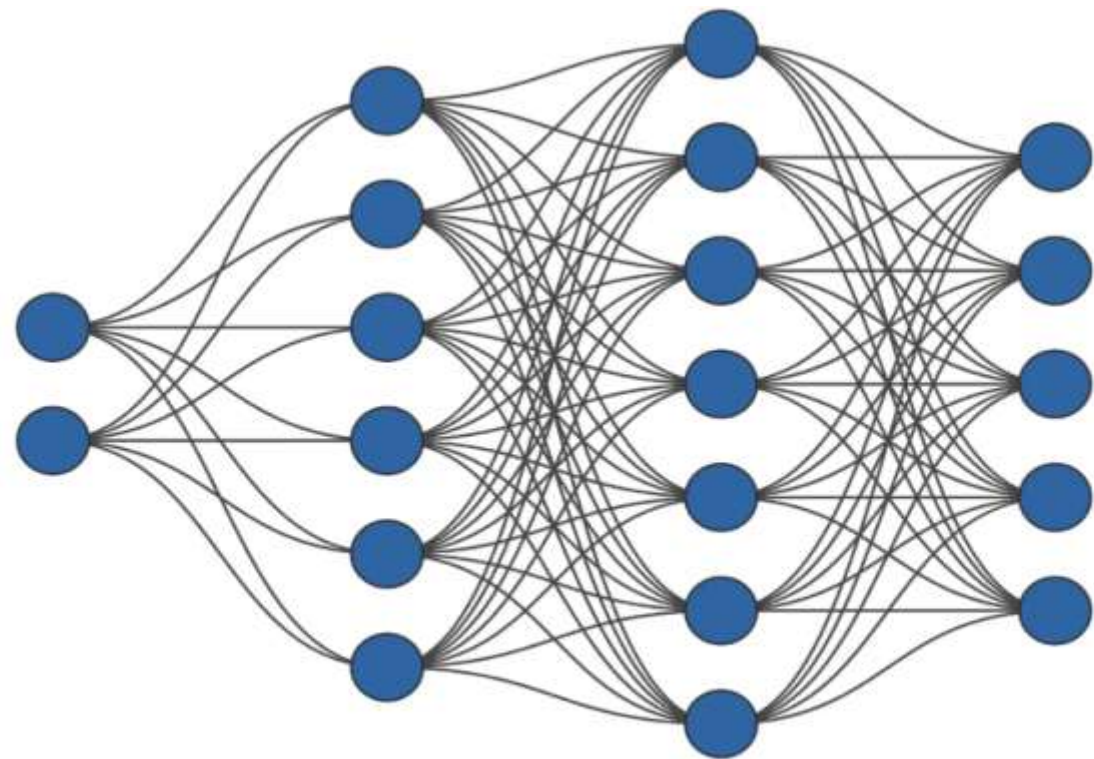
- 神经网络的基本原理与架构
- 基于Edge Impulse的神经网络设计
- Arduino神经网络语音识别实验



神经网络的基本原理与架构

■ 神经网络基本原理

- 神经网络是一种模仿生物神经系统的计算模型，其灵感来源于人脑中神经元的结构和工作方式。
- 神经网络通常由多层神经元组成，包括输入层、隐藏层和输出层。
- 神经网络训练过程包括前向传播、损失函数计算和反向传播。



Input Layer $\in \mathbb{R}^2$

Hidden Layer $\in \mathbb{R}^6$

Hidden Layer $\in \mathbb{R}^7$

Output Layer $\in \mathbb{R}^5$



神经网络基础知识

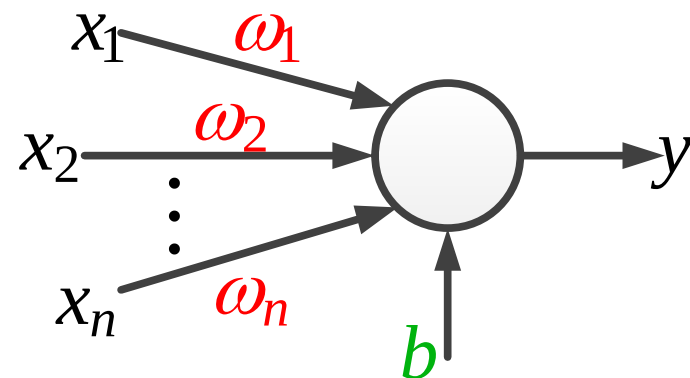
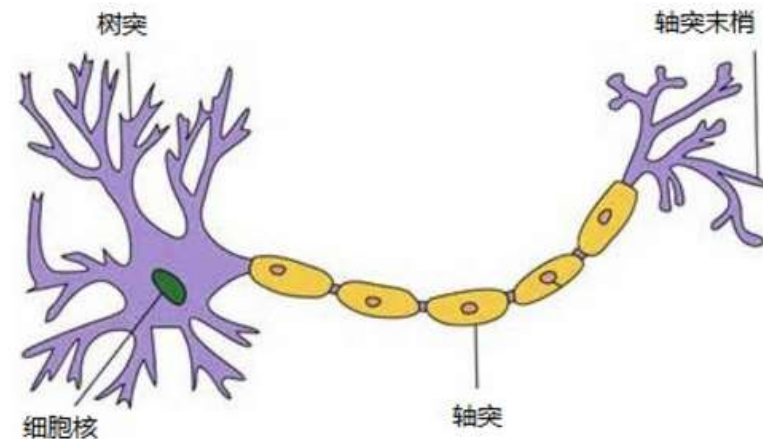
■ 神经元

- 形成神经网络的基本单元
- 处理输入并产生输出，而输出被发送到其他神经元用于进一步处理，或者作为最终输出结果

■ 权重、偏置

- 以下模型中， $x_1, x_2, \dots, x_n$ 表示某一神经元的 n 个输入
- ω_i 表示第 i 个输入的连接强度，称为权重
- b 另一个被应用于输入的线性分量，被称为偏置
- 经神经元处理后，输出 y

$$y = \sum_{i=1}^N \omega_i x_i + b$$





神经网络基础知识

■ 激活函数

➤ 激活函数将输入信号转换为输出信号，应用激活函数后的输出为 $f(\omega x + b)$
其中， $f(\cdot)$ 就是激活函数

➤ 常用激活函数

- Sigmoid

$$f(x) = \frac{1}{1 + e^{-x}}$$

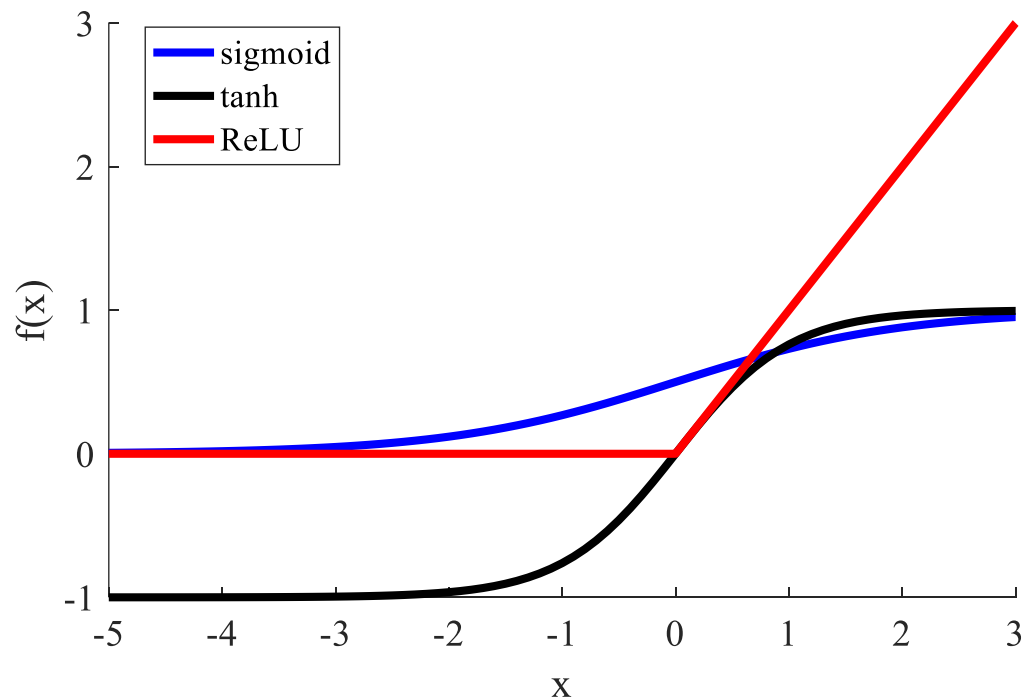
- Tanh

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- ReLU

$$f(x) = \max(0, x)$$

.....





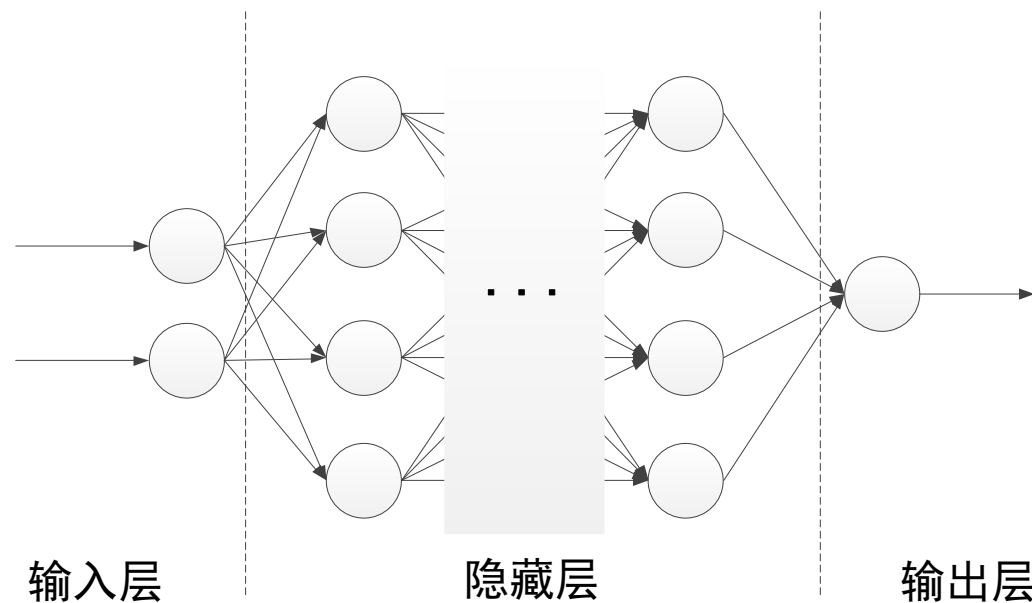
神经网络基础知识

■ 输入/输出/隐藏层

- 输入层是接收输入的一层，本质上是网络的第一层；
- 输出层是生成输出的一层，也是网络的最后一层；
- 隐藏层是处理层，对传入数据执行特定任务并将其生成的输出传递到下一层的那些层；

■ 多层感知机 (Multilayer Perceptron, MLP)

- 同一层神经元间无连接；
- 相邻两层神经元之间全连接；
- 信息传递有方向性；





深度学习基础知识

■ 损失函数

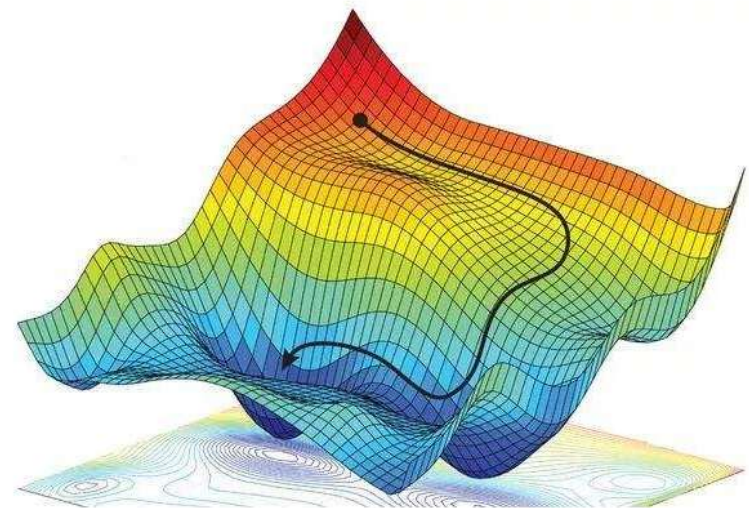
- 当建立一个网络时，试图将网络输出预测尽可能地接近实际值，通过损失函数来衡量网络的准确性

■ 学习方式

- 设置损失函数最小化处理为目标函数进行参数学习，典型方法为随机梯度下降

■ 学习率

- 下降到误差函数的最小值的速率；
- 该值过大，会错过最优结果，该值过小，网络学习速度较慢；



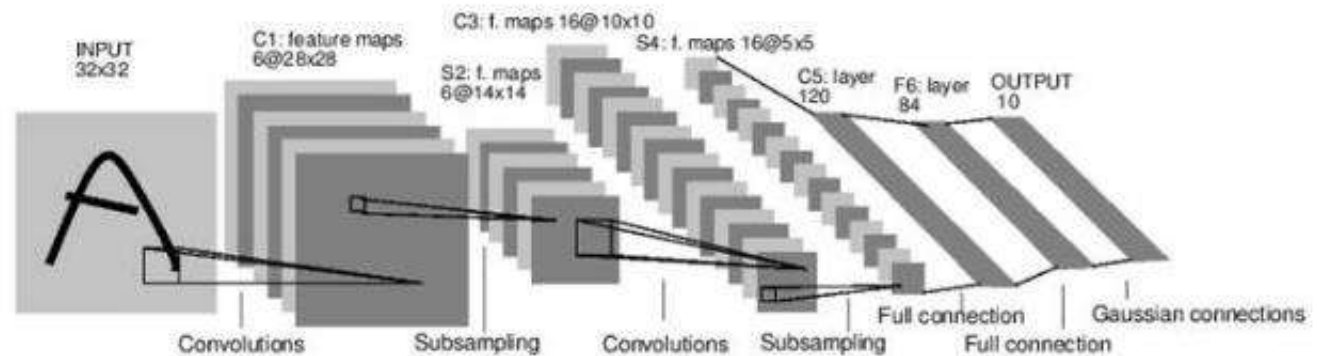


典型神经网络模型——CNN

■ 卷积神经网络 (Convolutional neural network, CNN)

- CNN是一种前馈神经网络，它的人工神经元可以覆盖部分周围单元，对于大型图像处理有出色表现；
- CNN的基本结构包括：卷积层，池化层
 - 卷积层：每个神经元的输入与前一层的局部接收域相连，并提取该局部的特征。
 - 池化层：目的是进行降采样，减小后续数据处理量；

典型CNN应用：
手写字母识别





典型神经网络模型——CNN

■ 卷积 (特征提取)

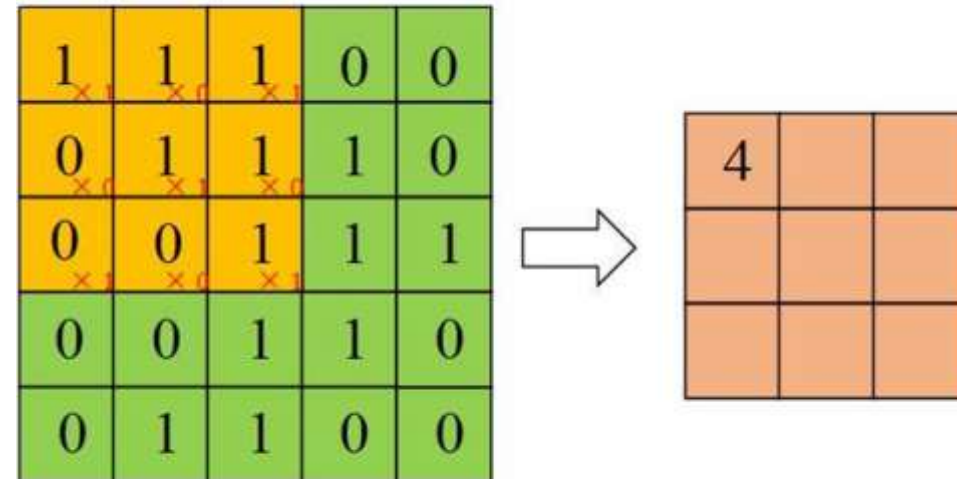
$$Y^{(l)}(s,t) = f \left(\sum_{u=0}^{k-1} \sum_{v=0}^{k-1} X^{(l-1)}(s+u, t+v) \times W^{(l)}(u,v) + b^{(l)} \right)$$

■ 池化 (降采样)

$$Y^{(l)}(s,t) = \max \left(X^{(l-1)}(u,v) \right)$$

■ softmax分类

$$y = \begin{pmatrix} P(t=1 | x; W) \\ P(t=2 | x; W) \\ \vdots \\ P(t=10 | x; W) \end{pmatrix} = \frac{1}{\sum_{i=1}^{10} \exp(W_i^T x)} \begin{pmatrix} \exp(W_1^T x) \\ \exp(W_2^T x) \\ \vdots \\ \exp(W_{10}^T x) \end{pmatrix}$$

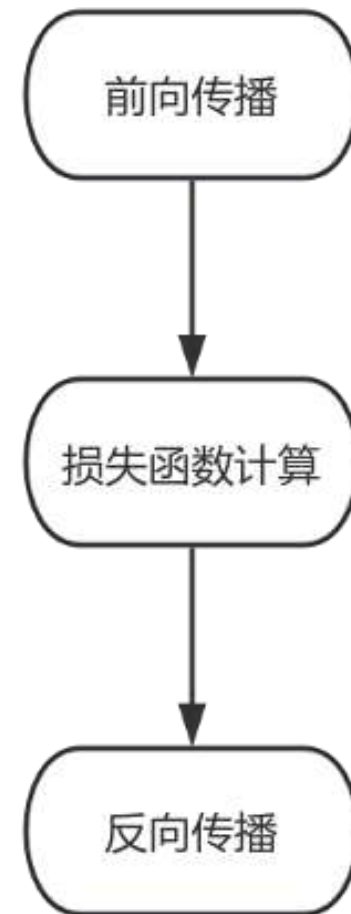




神经网络的基本原理与架构

■ 神经网络的训练过程

- 前向传播：输入数据从**输入层**逐层传递到**输出层**，每层的神经元根据前一层的输出和权重进行加权求和和激活函数转换，最终得到输出层的预测结果
- 损失函数用于衡量预测结果与真实标签之间的差异，常见的损失函数包括均方误差和交叉熵损失。
- 反向传播是神经网络训练的核心算法，通过计算损失函数对每个权重的梯度，从**输出层**向**输入层**逐层反向传播，更新网络的权重和偏置。





神经网络的基本原理与架构

■ 基于神经网络的语音识别系统实现

➤ 数据收集

- 需构建覆盖多语种、多方言及多场景的语音数据库，加性噪声注入（白噪声、背景人声）

➤ 特征提取

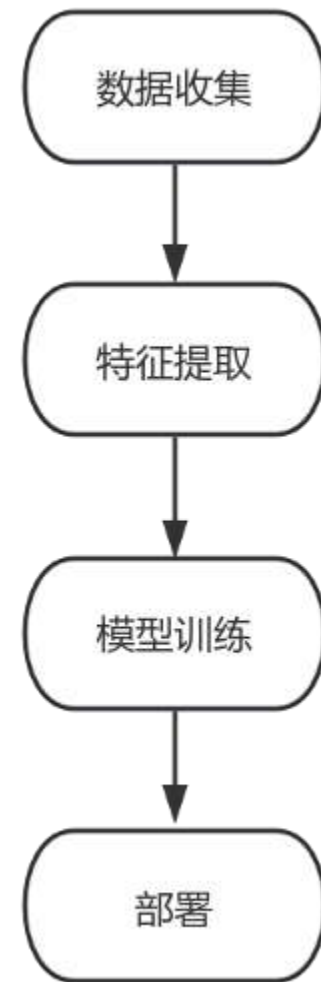
- 传统声学特征提取，采用 MFCC（Mel 频率倒谱系数）
- 通过卷积神经网络（CNN）自动提取高阶特征

➤ 模型训练

- 损失函数：损失函数与交叉熵联合优化

➤ 部署

- 量化训练：将 FP32 模型转换为 INT8 精度（精度损失 $< 1\%$ ）
- 嵌入式设备：TensorFlow Lite 转换模型



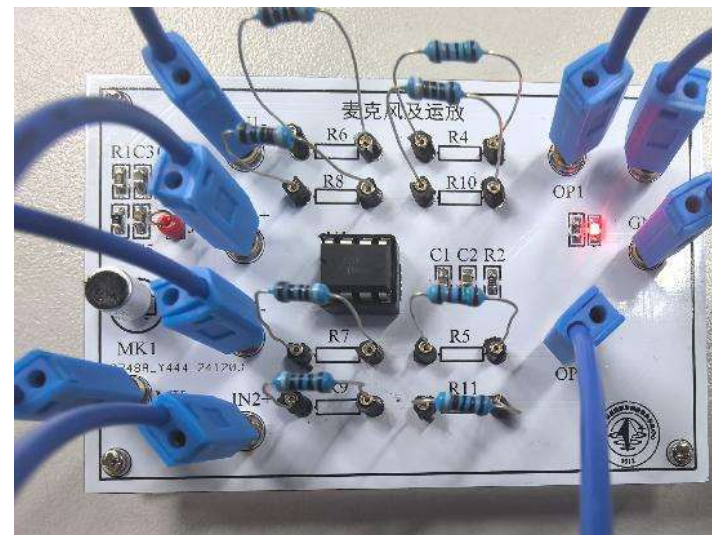


实验一：构建语音数据

■按图示方式连接音频放大电路（与上次实验相同）

- 传声器输出MK1连接到第一级放大器反相输入端IN1-
- 将ESP32的3.3V_OUT引脚接到第一级同相输入端IN1+
- 第一级输出OP1连接到第二级放大器反相输入端IN2-
- 将ESP32的3.3V_OUT引脚接到第二级同相输入端IN2+
- 将OP2连接到ESP32板的A0端口

■将speechRecognition.ino验证上传至ESP32C3中

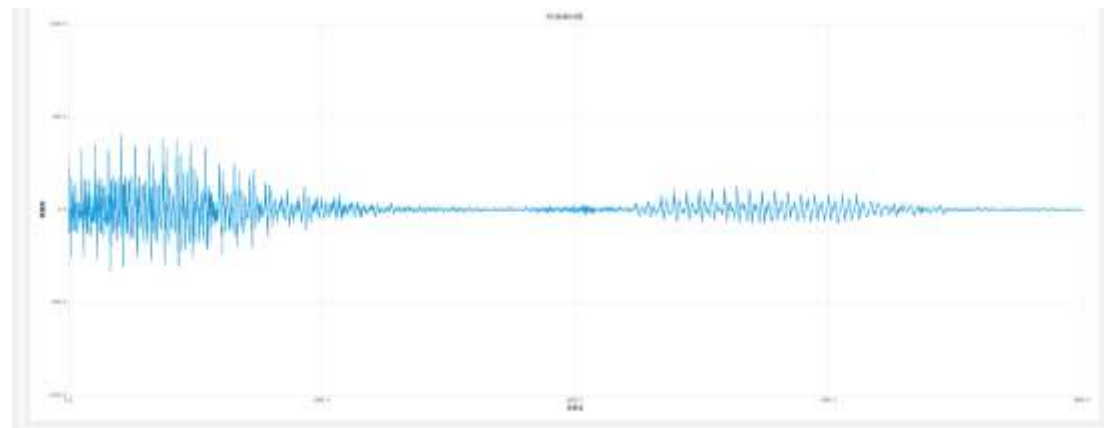




实验一：构建语音数据

■ 打开上位机SpeechLearn.exe

- 在语音采集界面选择【115200】波特率和对应的COM口，点击【Connect】连接ESP32C3单片机
- 点击【开始采集】，随便说几句话后，观察是否有波形图，点击【停止采集】，播放存储wav文件，验证语音采集功能正常

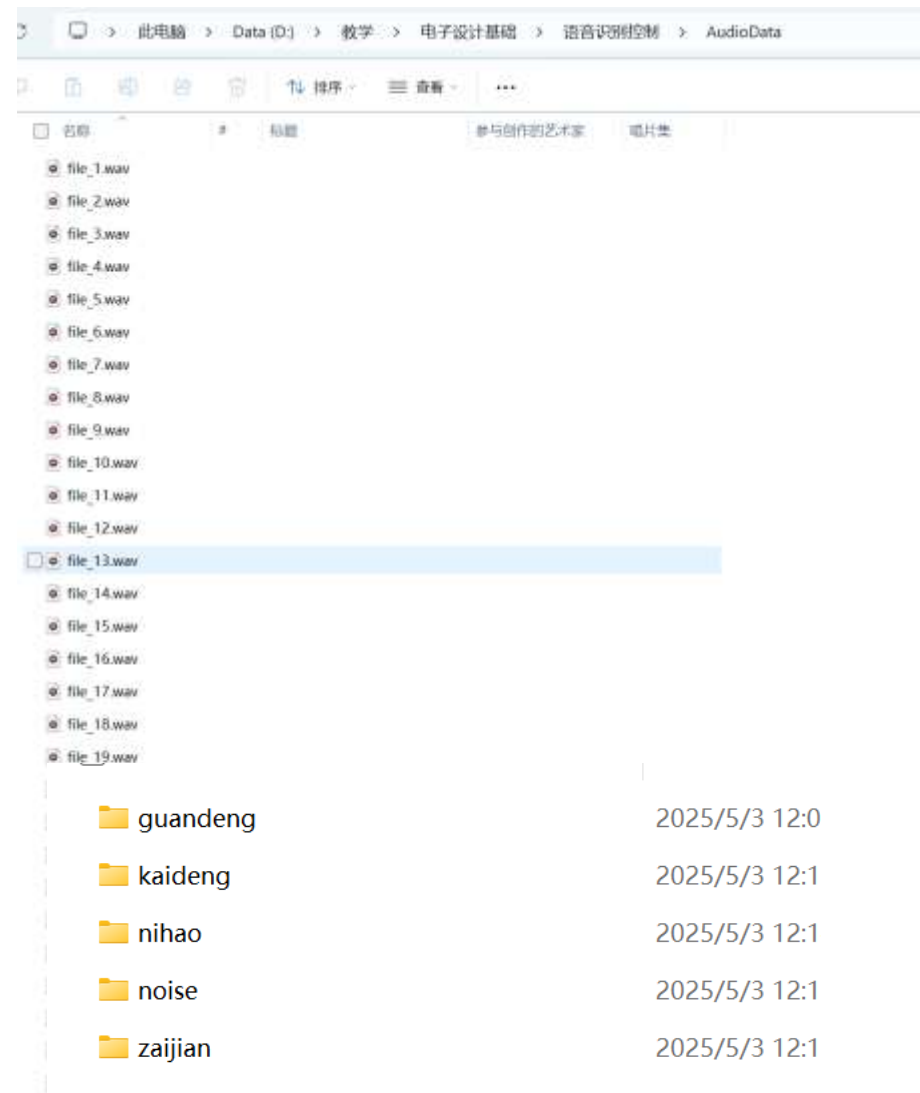




实验一：构建语音数据

■ 使用上位机SpeechLearn.exe

- 将AudioData文件夹删除
- 点击【开始采集】，发出各种噪音20遍以上，保证每次文件记录一次“噪声”指令，点击【采集结束】，将新生成的AudioData文件夹重命名为noise，并复制到前面实验中自己采集的语音文件夹MyData
- 采集结束后Disconnect串口，并关闭上位机软件
- 在SpeechData中已采集了“开灯、关灯、你好、再见、噪声”5组语音数据，每组120个语音文件，
- 可以将MyData中的语音文件合并到对应文件夹中，形成规模更大，多样性更强的数据集





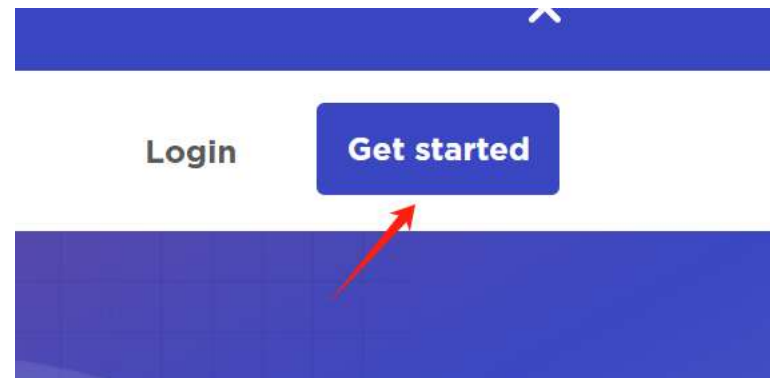
基于Edge Impulse的神经网络设计



EDGE IMPULSE

■ Edge Impulse简介

- Edge Impulse是一个用户友好的开发平台，允许开发者在云端**构建和训练**机器学习模型，并将模型**部署**到边缘设备上运行
- Edge Impulse涵盖了从**数据收集、特征提取到模型训练和部署**的完整流程
- 访问Edge Impulse官网并注册
- <https://edgeimpulse.com>



or

Email

Username

Password

What describes your role at your company? (optional)

Let us know you're human

☐ 确认您是真人

 CLOUDFLARE

☐ I accept the Privacy Policy, Community Terms of Service, and Responsible AI License.

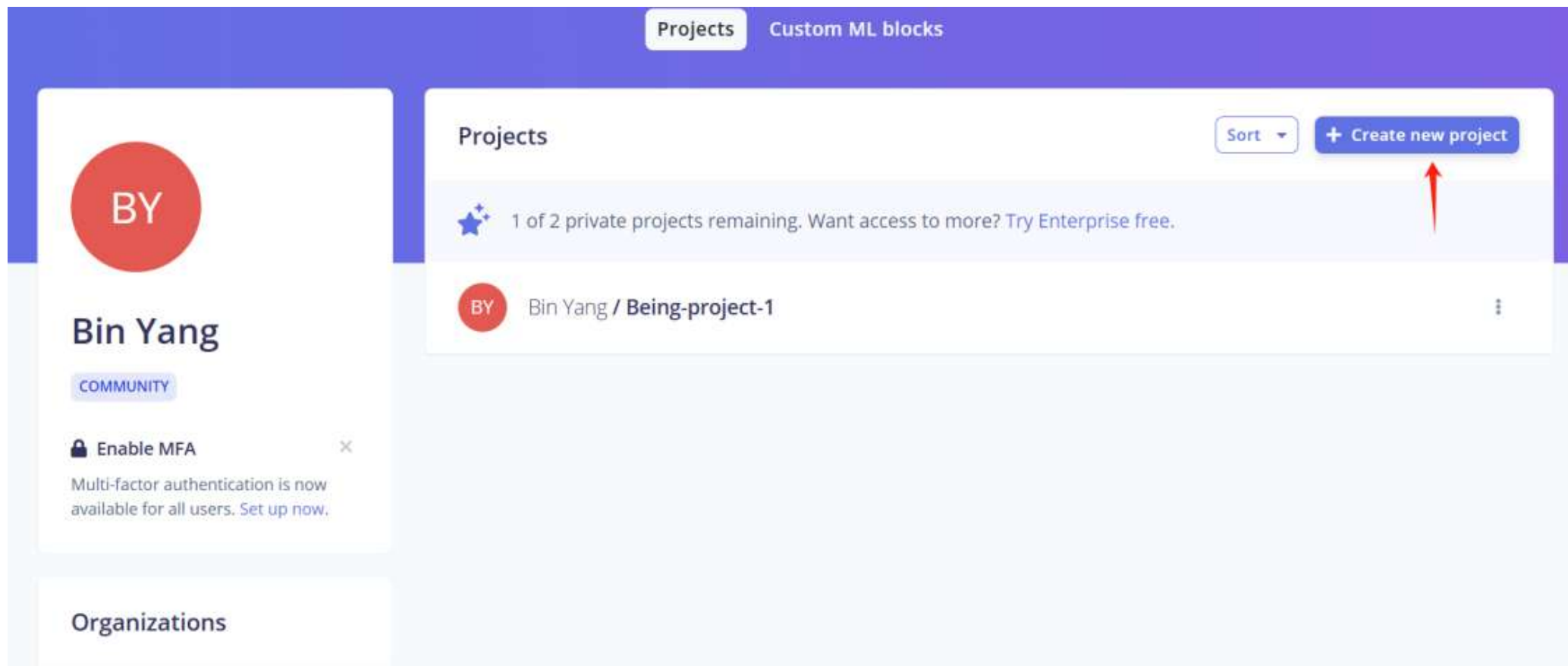
Already have an account? Log in



基于Edge Impulse的神经网络设计

■ 创建新工程

- 登录 Edge Impulse 平台,点击【Create new project】创建一个新的项目,选择“语音”作为数据类型。

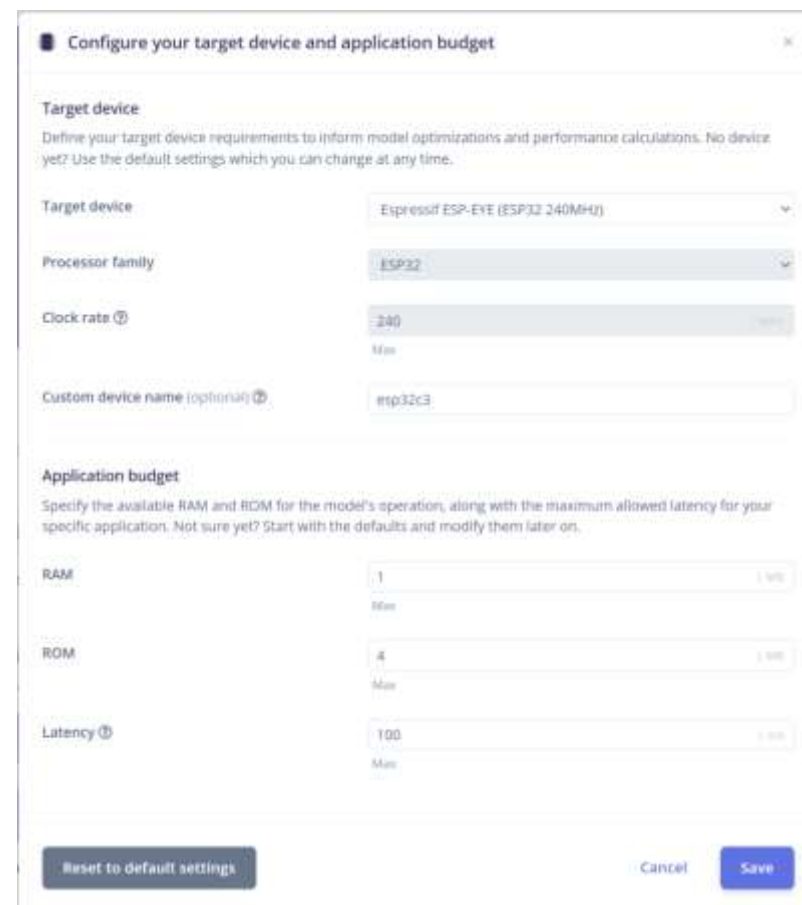
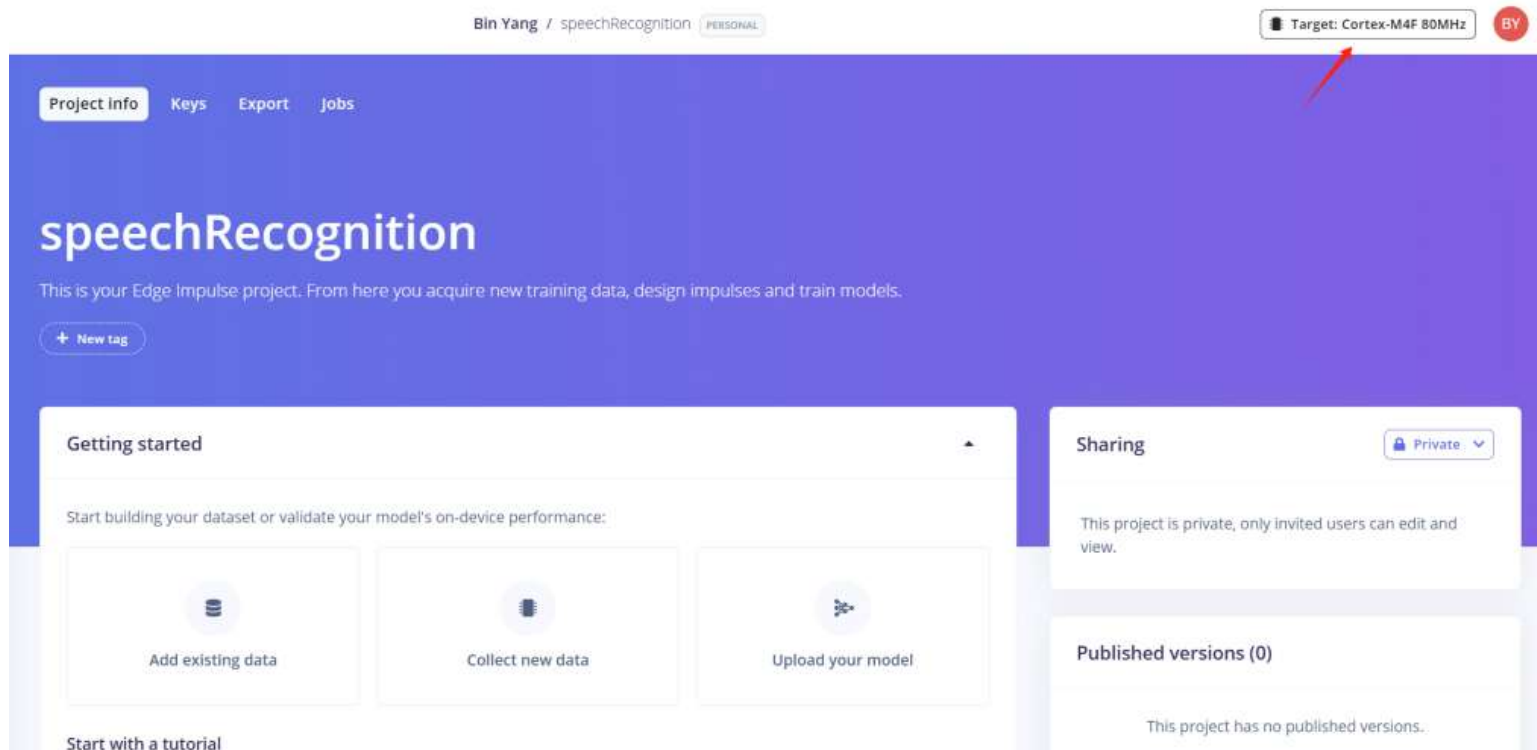




基于Edge Impulse的神经网络设计

■ 在右上角选择目标器件

➤ 目标设备选择**Espressif ESP-EYE(ESP32 240MHz)**，其他参数如下方右图所示

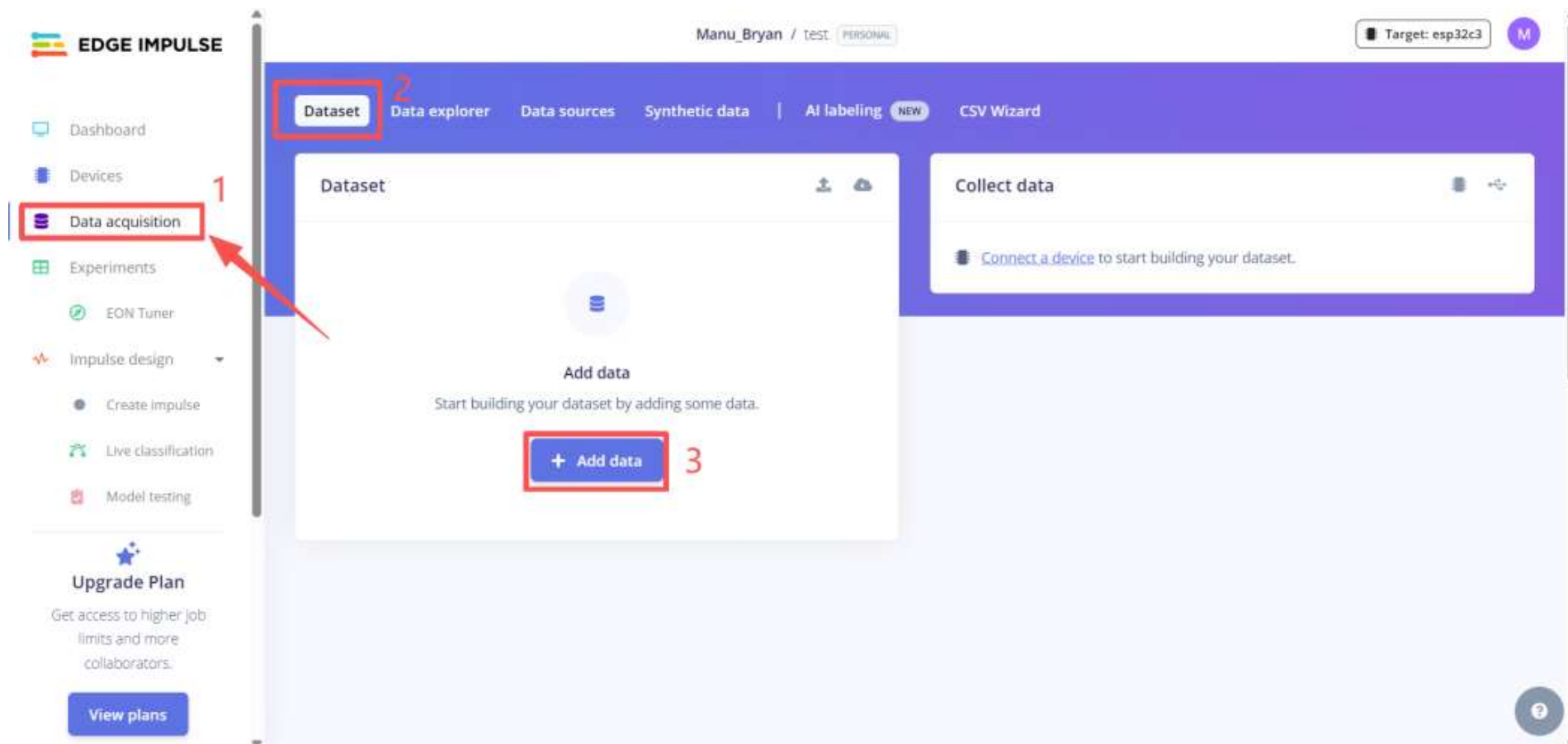




基于Edge Impulse的神经网络设计

■ 上传数据集

- 进入Data Acquisition模块，选择Dataset，点击【Upload data】上传采集的所有音频数据文件





基于Edge Impulse的神经网络设计

■ 上传数据集

- 从采集好的【kaideng】【guandeng】【nihao】【zaijian】【noise】中选择kaideng文件夹
- 使用自动拆分成训练集和测试集
- Label中选择Enter label
输入: kaideng
- 点击Upload data等待上传完毕
- 将其他几条指令数据集分别上传

Upload data

You can upload CBOR, JSON, CSV, Parquet, WAV, JPG, PNG, AVI or MP4 files. You can also upload an annotation file named "info.labels" with your data to assign bounding boxes, labels, and/or metadata. View [Uploader docs](#) to learn more. Alternatively, you can use our [Python SDK](#) to programmatically ingest data in various formats, such as pandas or numpy.

For CSV and Parquet files, [configure the CSV Wizard](#) to define how your files should be processed before uploading files.

Upload mode

☐ Select individual files ⓘ

☒ Select a folder ⓘ 1

Select files

未选择任何文件 2

Upload into category

☒ Automatically split between training and testing ⓘ 3

☐ Training

☐ Testing

Label

☐ Infer from filename ⓘ

☐ Leave data unlabeled ⓘ

☒ Enter label: 4

[< Back](#)

[Upload data](#) 5



基于Edge Impulse的神经网络设计

■ 上传数据集

- 从采集好的【kaideng】【guandeng】【nihao】【zaijian】【noise】中选择kaideng文件夹
- 使用自动拆分成训练集和测试集
- Label中选择Enter label
输入: kaideng
- 点击Upload data等待上传完毕
- 将其他几条指令数据集分别上传





基于Edge Impulse的神经网络设计

■ 上传数据集

- 从采集好的【kaideng】【guandeng】【nihao】【zaijian】【noise】中选择kaideng文件夹
- 使用自动拆分成训练集和测试集
- Label中选择Enter label
输入: kaideng
- 点击Upload data等待上传完毕
- 将其他几条指令数据集分别上传

For CSV and Parquet files, configure the CSV wizard to define how your files should be processed before uploading files.

Upload mode

☐ Select individual files ⓘ

☒ Select a folder ⓘ

Select files

选择文件 未选择任何文件

Upload into category

☒ Automatically split between training and testing ⓘ

☐ Training

☐ Testing

Label

☐ Infer from filename ⓘ

☐ Leave data unlabeled ⓘ

☒ Enter label:

kaideng

Upload output

```
[103/120] uploading file_83.wav OK
[104/120] uploading file_87.wav OK
[105/120] uploading file_82.wav OK
[106/120] uploading file_86.wav OK
[107/120] uploading file_84.wav OK
[108/120] uploading file_80.wav OK
[109/120] uploading file_88.wav OK
[110/120] uploading file_90.wav OK
[111/120] uploading file_91.wav OK
[112/120] uploading file_9.wav OK
[113/120] uploading file_93.wav OK
[114/120] uploading file_92.wav OK
[115/120] uploading file_95.wav OK
[116/120] uploading file_94.wav OK
[117/120] uploading file_96.wav OK
[118/120] uploading file_98.wav OK
[119/120] uploading file_97.wav OK
[120/120] uploading file_99.wav OK
```

Done. Files uploaded successfully: 120. Files that failed to upload: 0.

100% completed

[< Back](#)

[Upload data](#)



基于Edge Impulse的神经网络设计

■ 上传数据集

DATA COLLECTED

10m 0s

TRAIN / TEST SPLIT

80% / 20%



Dataset



Training (479) Test (121)



SAMPLE NAME	LABEL	ADDED	LENGTH	
file_97	noise	Today, 12:48:55	1s	
file_99	noise	Today, 12:48:55	1s	
file_98	noise	Today, 12:48:55	1s	
file_93	noise	Today, 12:48:55	1s	
file_94	noise	Today, 12:48:55	1s	
file_92	noise	Today, 12:48:55	1s	
file_91	noise	Today, 12:48:55	1s	
file_9	noise	Today, 12:48:55	1s	
file_89	noise	Today, 12:48:55	1s	
file_88	noise	Today, 12:48:55	1s	
file_86	noise	Today, 12:48:54	1s	





基于Edge Impulse的神经网络设计

■ 设置处理框架

➤ 单击【Add a processing block】添加**数据预处理模块**

- MFCC
- MFE

The screenshot displays the Edge Impulse web interface. On the left sidebar, the 'Impulse design' option is selected and highlighted with a red box, labeled with a red '1'. Below it, the 'Create impulse' button is also highlighted with a red box and labeled with a red '2'. The main workspace shows the 'Impulse #1' configuration. The 'Time series data' panel on the left is highlighted with a red box and labeled with a red '3'. It contains settings for 'Input axes' (audio), 'Window size' (1,000 ms), 'Window increase (stride)' (500 ms), 'Frequency (Hz)' (8000), and 'Zero-pad data' (checked). To the right of this panel are two dashed boxes: 'Add a processing block' (highlighted with a red box and labeled with a red '3') and 'Add a learning block'. Further right is the 'Output features' panel, which is green and has a checkmark icon. A 'Save impulse' button is located at the bottom right of the workspace. The footer shows the copyright notice: '© 2026 EdgeImpulse Inc. All rights reserved.'



基于Edge Impulse的神经网络设计

■ 设置处理框架

➤ 以MFCC为例，选择MFCC模块，点击【Add】

 Add a processing block ×

Did you know? You can [bring your own DSP code](#).

DESCRIPTION	AUTHOR	RECOMMENDED
Audio (MFCC) OFFICIALLY SUPPORTED Extracts features from audio signals using Mel Frequency Cepstral Coefficients, great for human voice.	Edge Impulse	4  
Audio (MFE) OFFICIALLY SUPPORTED Extracts a spectrogram from audio signals using Mel-filterbank energy features, great for both voice and non-voice audio.	Edge Impulse	 



基于Edge Impulse的神经网络设计

■ 设置处理框架

➤ 单击【Add a learning block】添加**模型训练模块**，选择Classification模块

EDGE IMPULSE

Manu_Bryan / exp11

Target: esp32c3

Impulse #1

An impulse takes raw data, uses signal processing to extract features, and then uses a learning block to classify new data.

Time series data

Input axes: audio

Window size: 1,000 ms.

Window increase (stride): 500 ms.

Frequency (Hz): 8000

Zero-pad data: ☒

Audio (MFCC)

Name: MFCC

Input axes (1): audio

Add a learning block

Output features

Save Impulse

Upgrade Plan

Get access to higher job limits and more collaborators.

View plans

<https://studio.edgeimpulse.com>



基于Edge Impulse的神经网络设计

■ 设置处理框架

➤ 点击【Add】添加后，点击【Save Impulse】保存设置

🔧 Add a learning block

Did you know? You can bring your own model in PyTorch, Keras or scikit-learn.

DESCRIPTION	AUTHOR	RECOMMENDED
Classification OFFICIALLY SUPPORTED Learns patterns from data, and can apply these to new data. Great for categorizing movement or recognizing audio.	Edge Impulse	Add

ing to extract features, and then uses a learning block to classify new data.

Audio (MFCC)

Name: MFCC

Input axes (1):

Signal: audio

Classification

Name: Classifier

Input features:

☒ MFCC

Output features: 5 (guandeng, kaideng, nihao, noise, zaijian)

Output features

5 (guandeng, kaideng, nihao, noise, zaijian)

Save Impulse

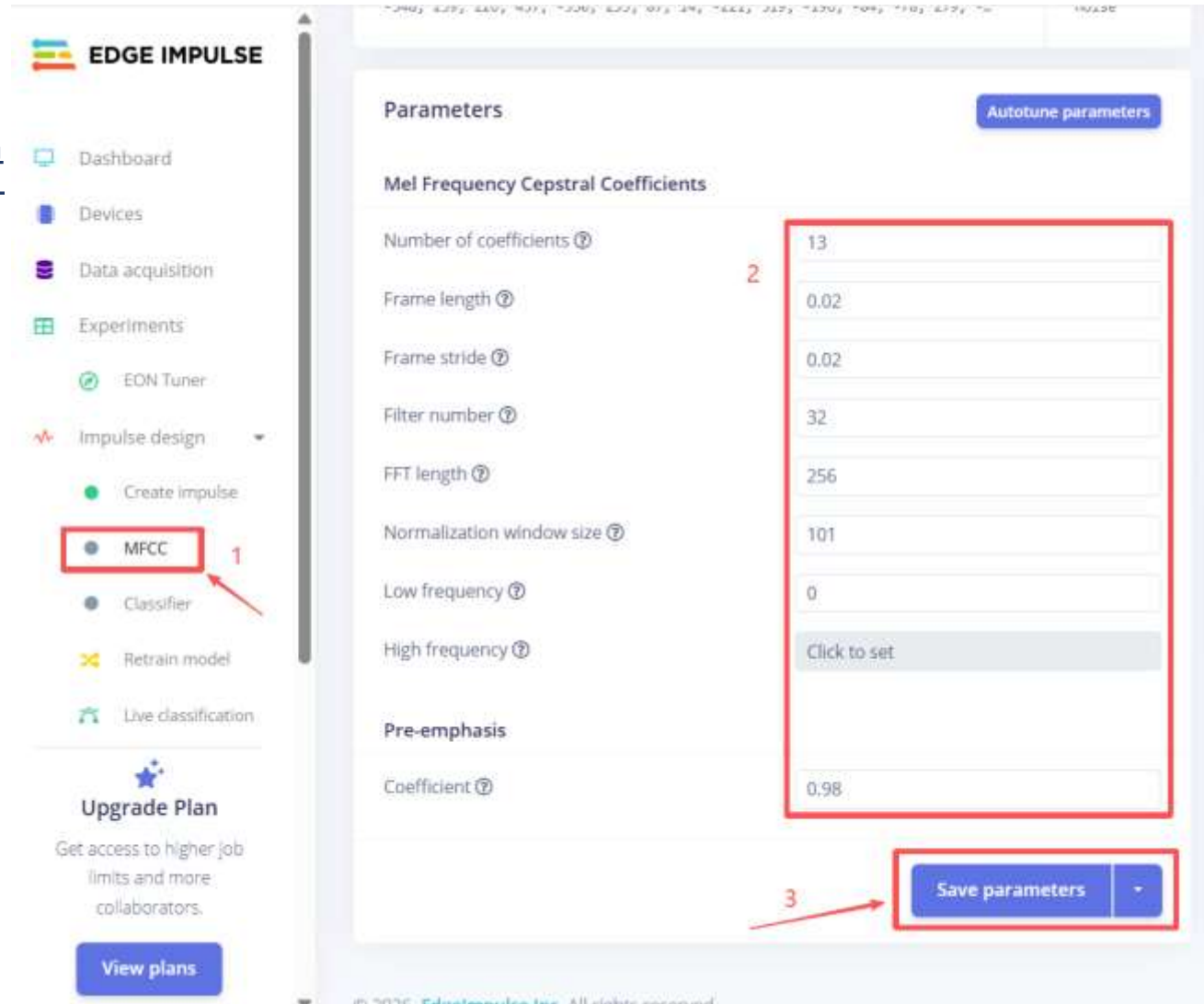
Add a processing block Add a learning block



基于Edge Impulse的神经网络设计

■ 设置预处理模块

- 点击【MFCC】
- 【Number of coefficients】设置为13
- 【Frame length】设置为0.02
- 【Frame stride】设置为0.02
- 【Filter number】设置为32
- 【FFT length】设置为256
- 【Normalization window size】设置为101
- 【Low frequency】设置为0
- 【Coefficient】设置为0.98

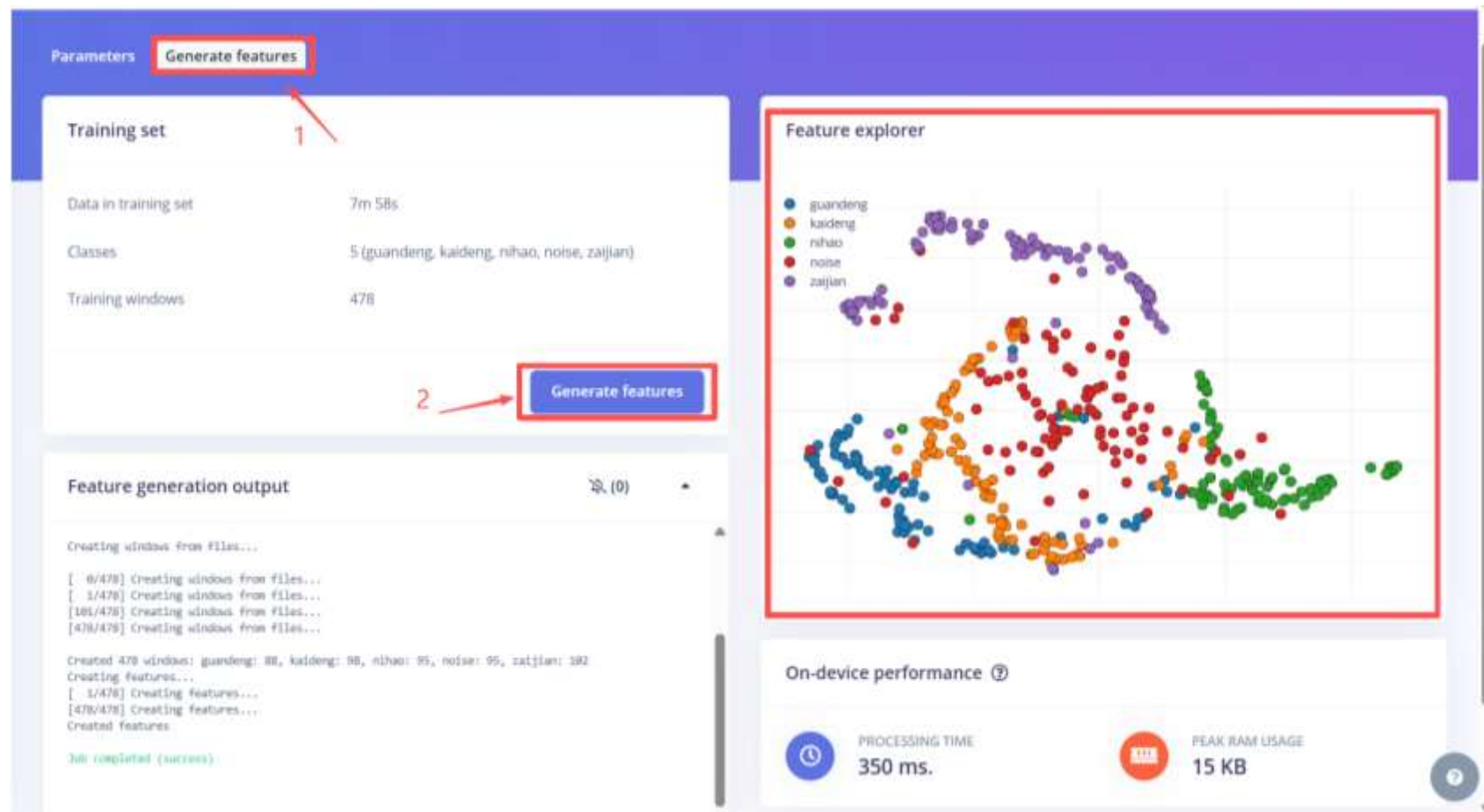




基于Edge Impulse的神经网络设计

■ 设置预处理模块

- 生成特征
- 等待处理完成
- 查看特征分布
- 注意处理时间

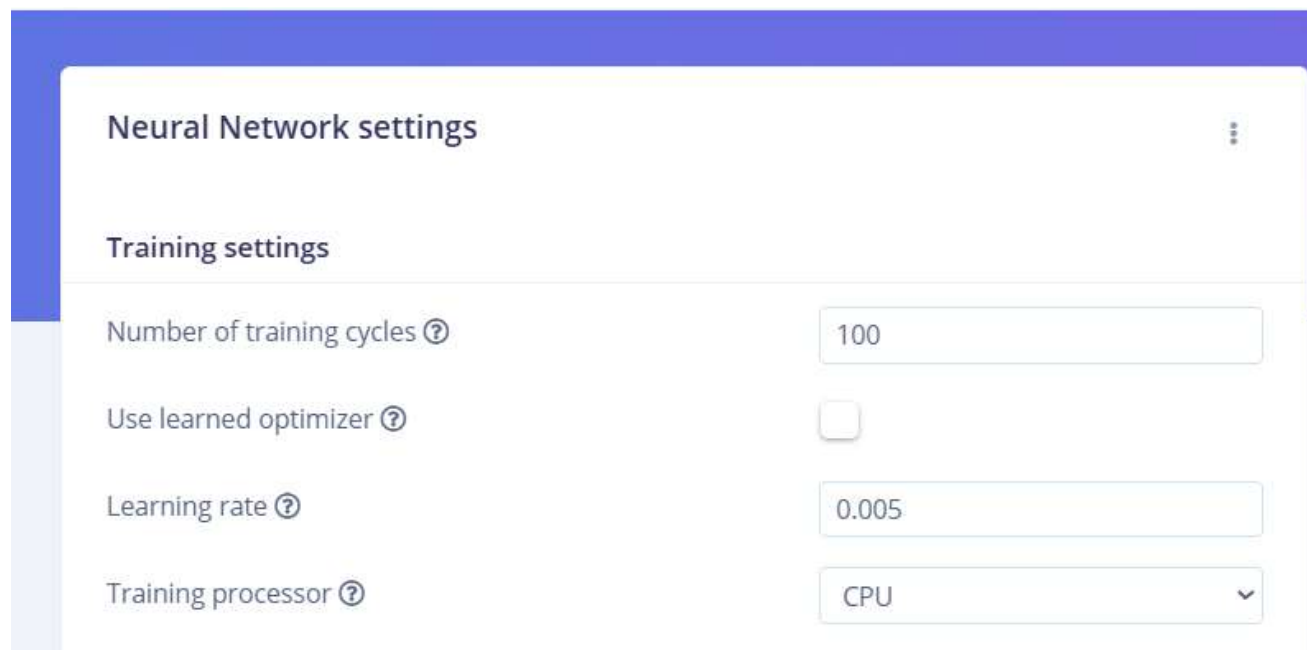
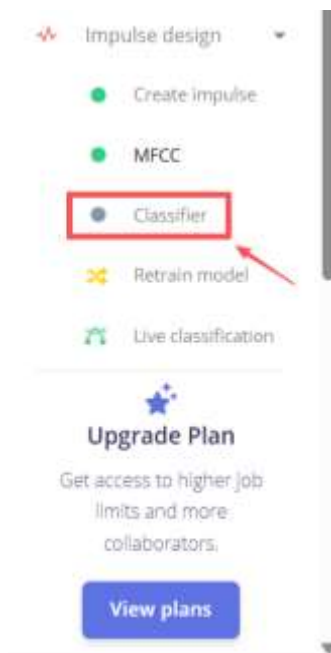




基于Edge Impulse的神经网络设计

■ 设置学习模块

- 点击【Classifier】
- 【Number of training cycles】设置为100
- 取消勾选【Use learned optimizer】
- Learning rate】设置为0.005
- 【Training processor】设置为CPU





基于Edge Impulse的神经网络设计

■ 设置学习模块

- Neural Network settings页面下的Neural network architecture部分
- 上层【1D conv / pool layer】设置参数为：8 filters, 3 kernel size, 1 layer
- 下层【1D conv / pool layer】设置参数为：16 filters, 3 kernel size, 1 layer
- 设置完成后点击下方【save&train】开始训练

Neural network architecture

Neural network Transfer learning Load preset...

Input layer (650 features)

Reshape layer (13 columns)

1D conv / pool layer (8 filters, 3 kernel size, 1 layer)

Dropout (rate 0.25)

1D conv / pool layer (16 filters, 3 kernel size, 1 layer)

Dropout (rate 0.25)

Flatten layer

Add an extra layer

Output layer (5 classes)

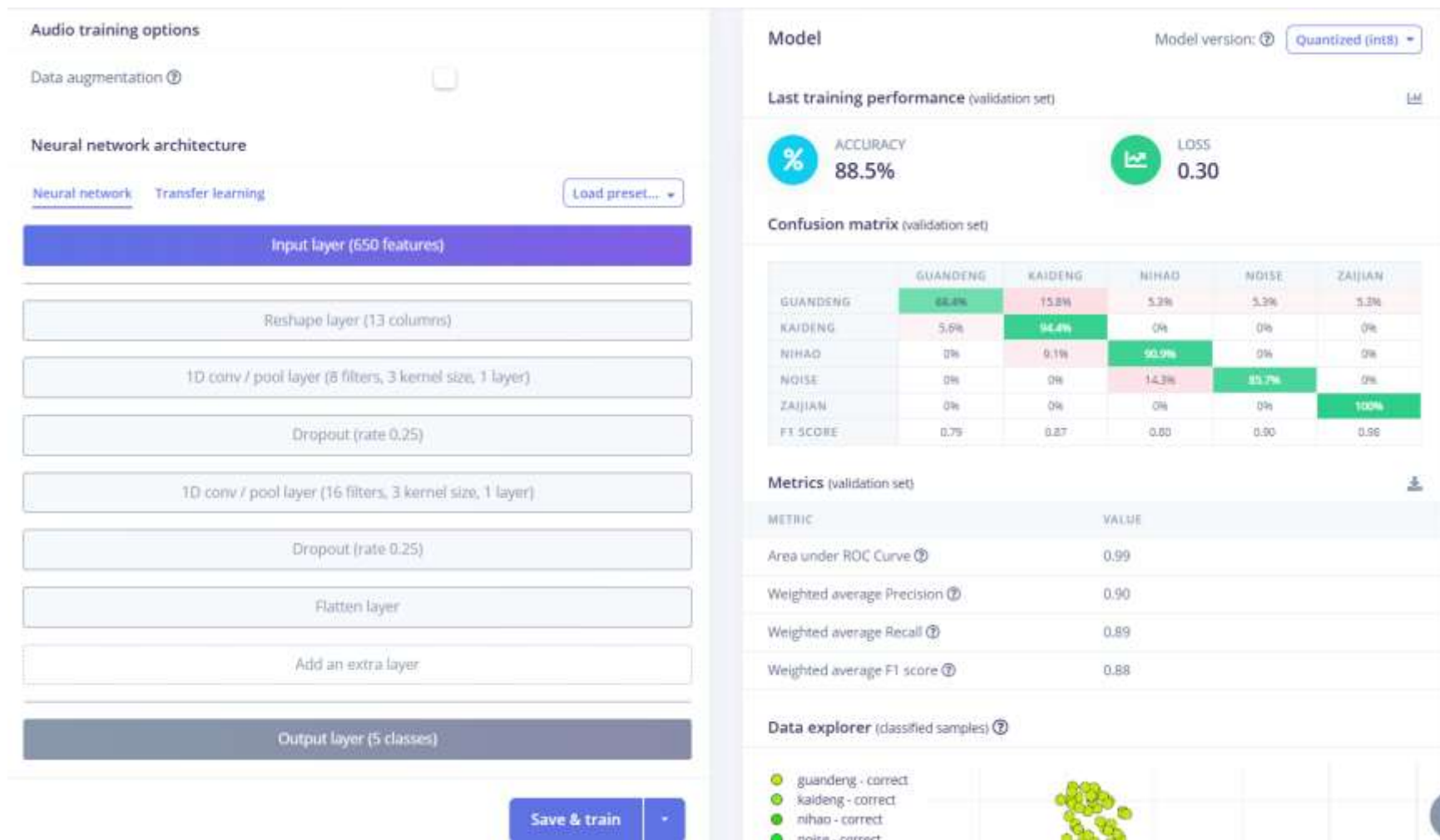
Save & train



基于Edge Impulse的神经网络设计

■ 设置学习模块

- 等待训练完成
- 查看模型精度
- 查看混淆矩阵
- 查看性能评测



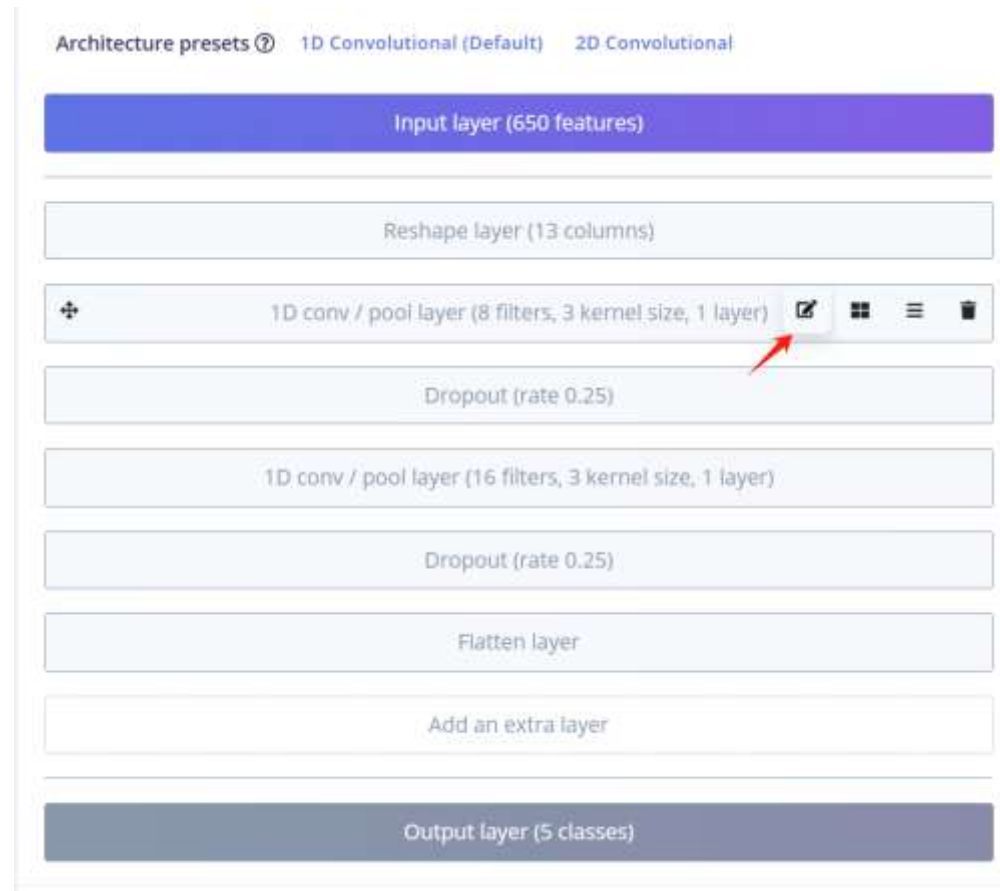


基于Edge Impulse的神经网络设计

■ 设置学习模块

➤ 修改网络模型

- 增加网络宽度——增加滤波器个数
- 增加感受野——增加kernel size
- 增加网络深度——增加内层数
- 添加额外层





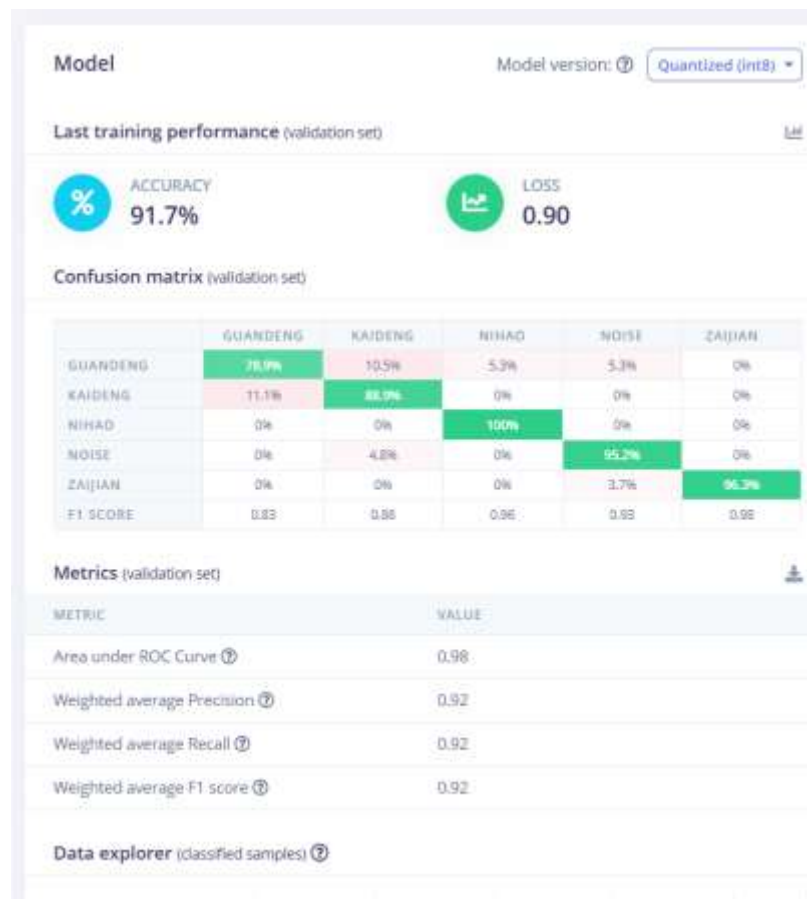
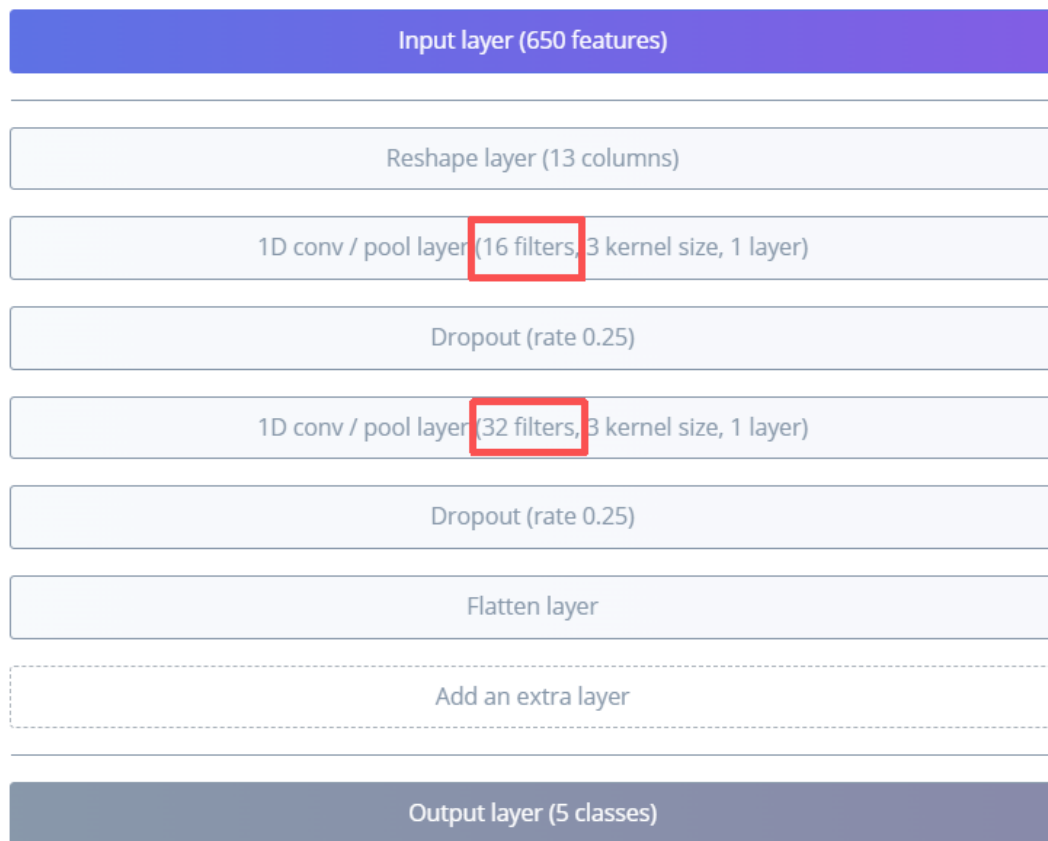
基于Edge Impulse的神经网络设计

■ 设置学习模块

➤ 修改网络模型

- 增加网络宽度——增加滤波器个数

- 将第一个卷积池化层的滤波器个数改为16
- 第二个卷积池化层的滤波器个数改为32
- 保存并重新训练模型，观察准确率是否提升





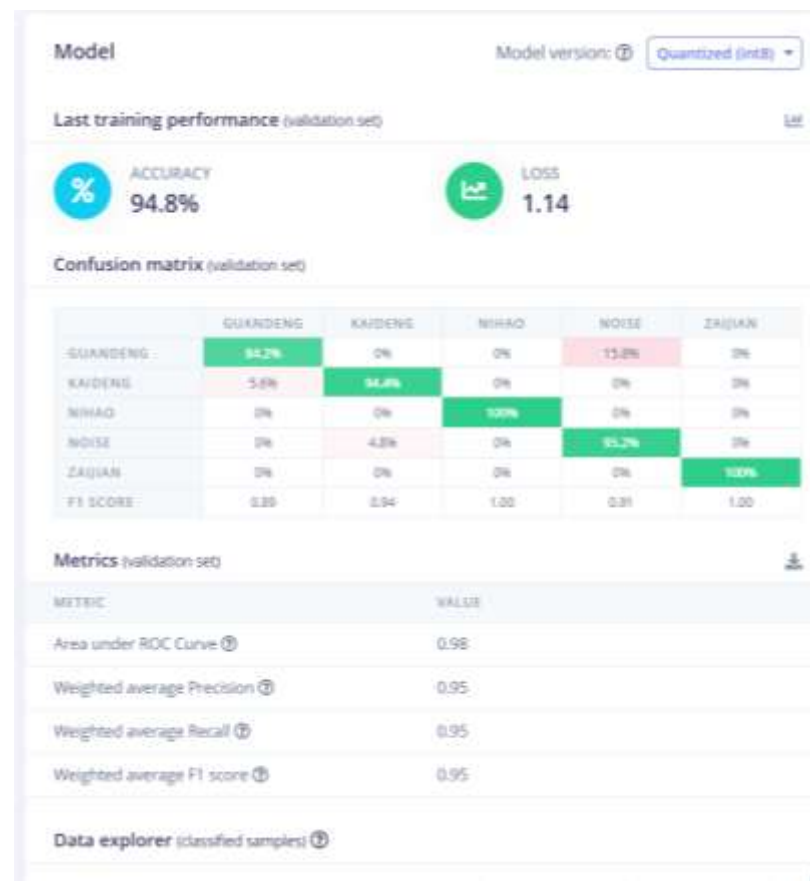
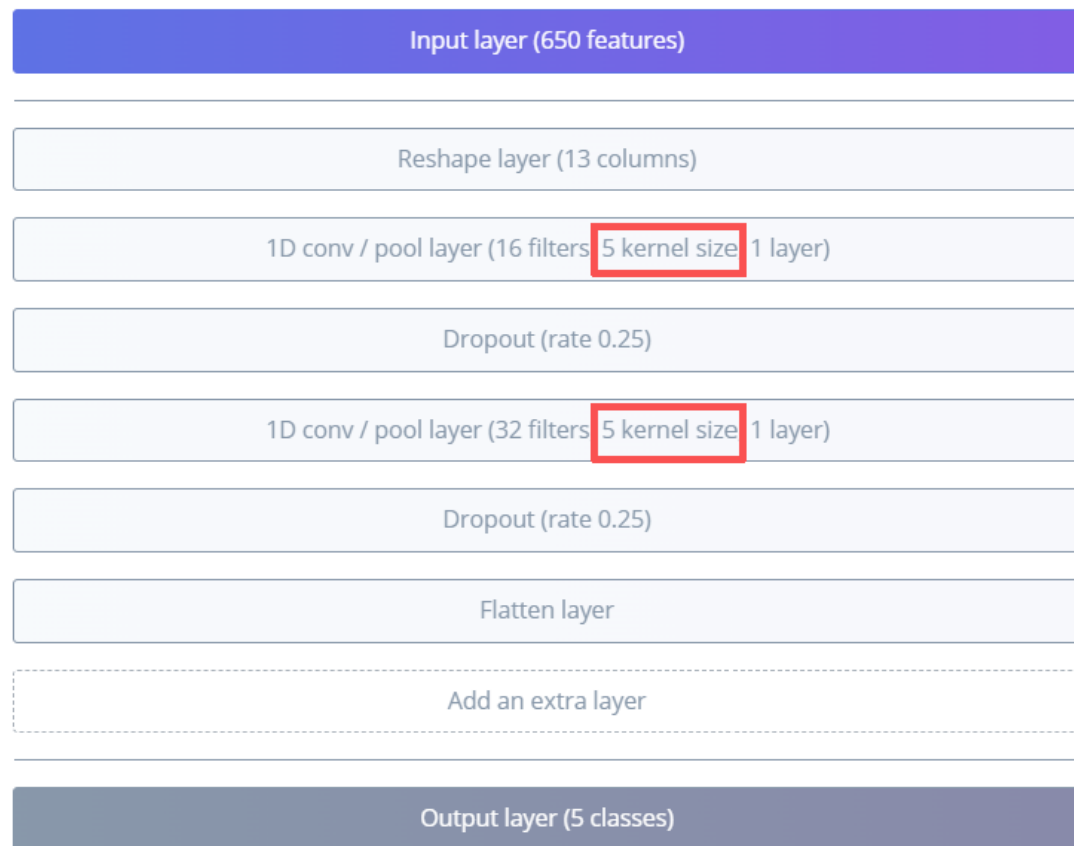
基于Edge Impulse的神经网络设计

■ 设置学习模块

➤ 修改网络模型

- 增加感受野——增加kernel size

- 将两个卷积池化层的kernel size改为5
- 保存并重新训练模型，观察准确率是否提升



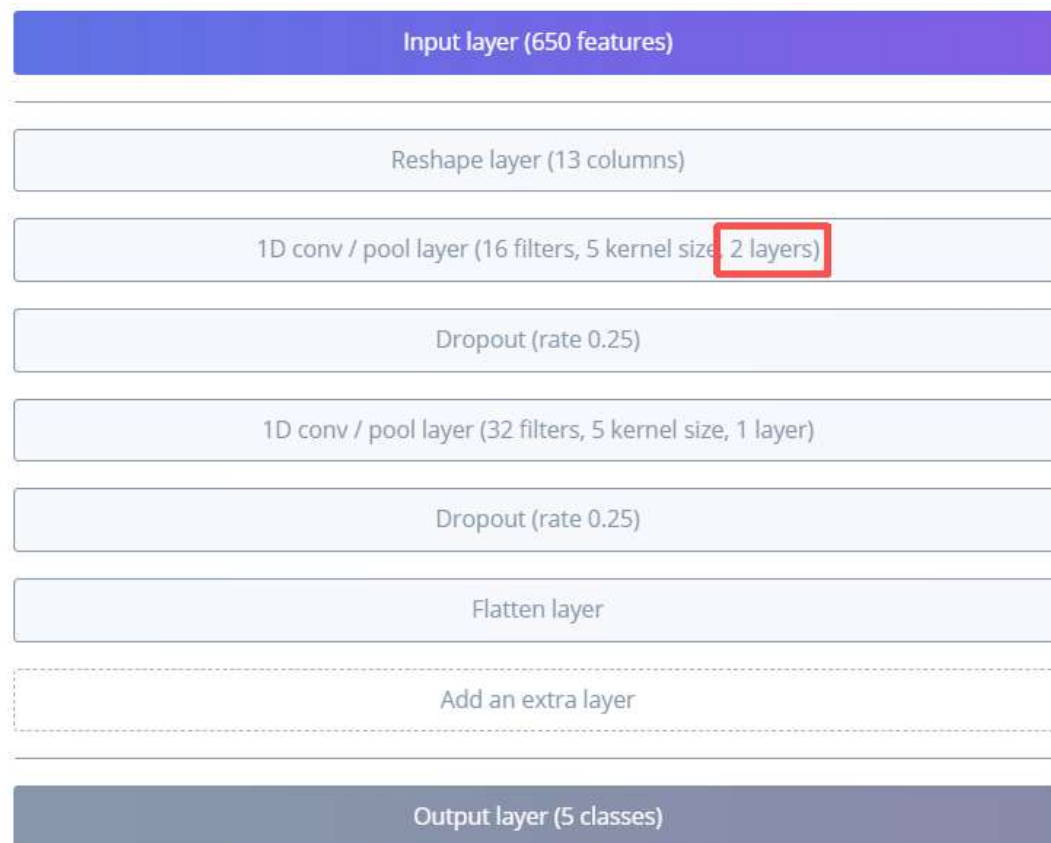


基于Edge Impulse的神经网络设计

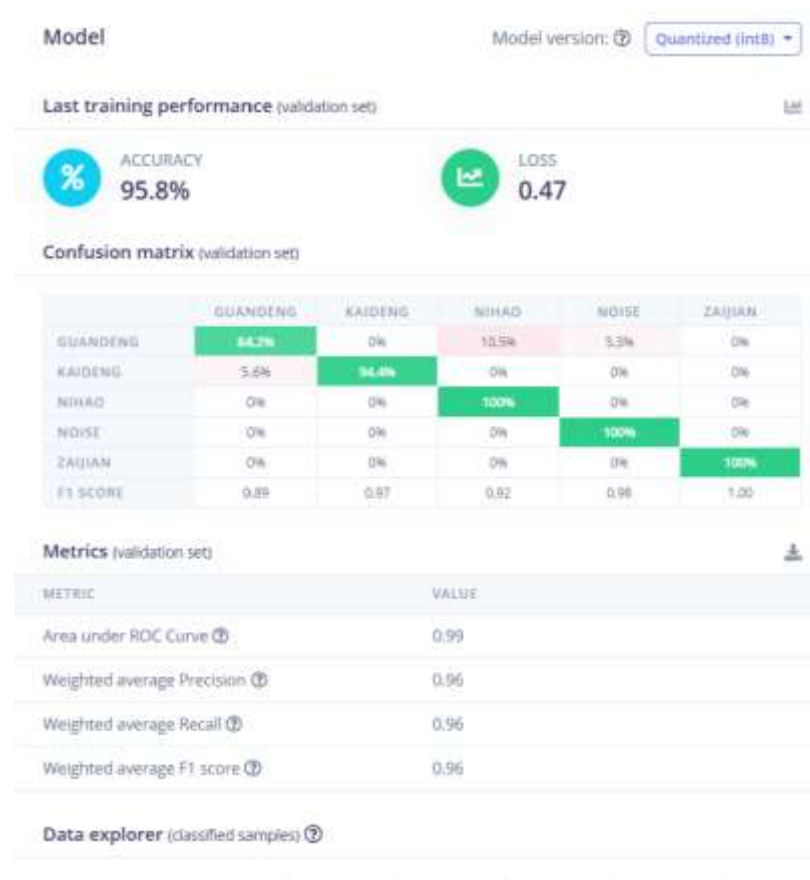
■ 设置学习模块

➤ 修改网络模型

- 增加网络深度——增加内层数



- 将第一个卷积池化层的内层数改为2
- 保存并重新训练模型，观察准确率是否提升





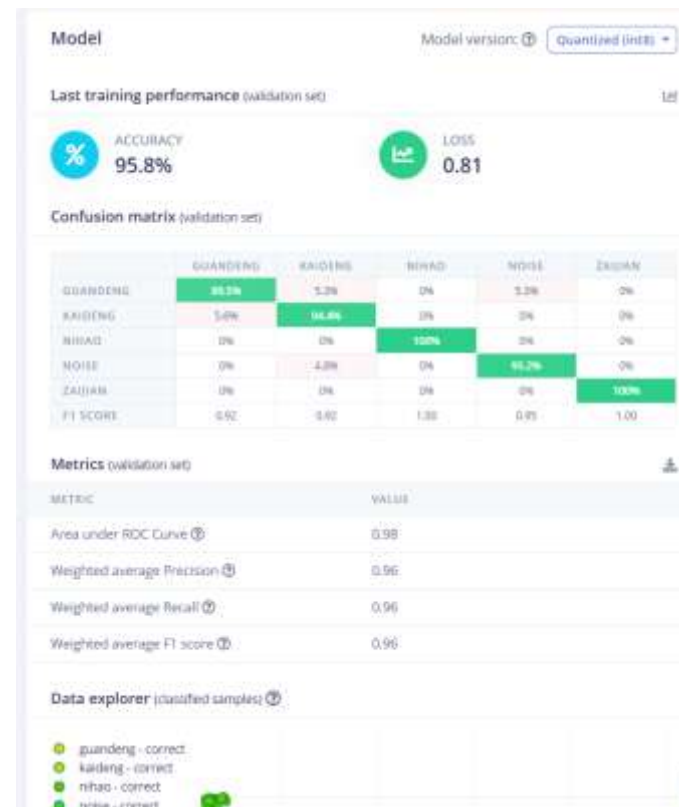
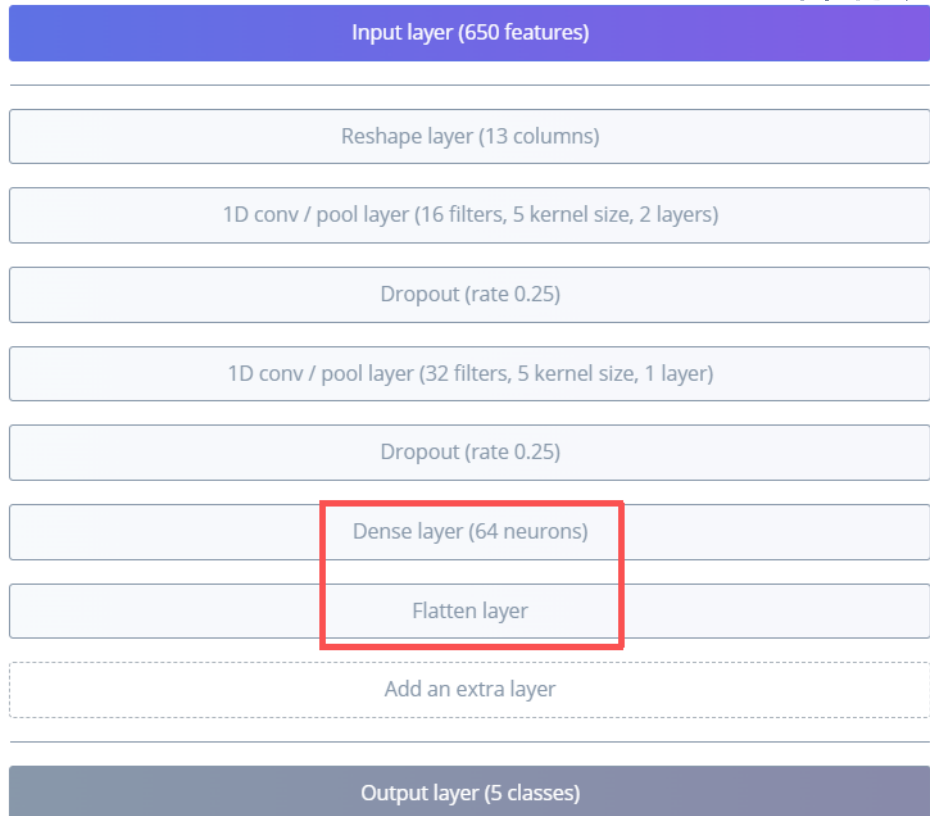
基于Edge Impulse的神经网络设计

■ 设置学习模块

➤ 修改网络模型

• 添加额外层

- 点击【Add an extra layer】
- 添加Dense layer
- 设置神经元个数为64
- 将网络架构中的Flatten layer与新增的Dense layer调换位置
- 保存并重新训练模型，观察准确率是否提升





基于Edge Impulse的神经网络设计

■ 设置学习模块

- 查看训练结果
- 查看推理资源消耗
 - 推理时间
 - 峰值RAM使用
 - FLASH使用

On-device performance ?

Engine: ?

EON™ Compiler (RAM optimized) ▼



INFERRING TIME

26 ms.



PEAK RAM USAGE

22.8K



FLASH USAGE

69.8K

Model

Model version: ?

Quantized (int8) ▼

Last training performance (validation set)



ACCURACY

96.9%



LOSS

0.07

Confusion matrix (validation set)

	GUANDENG	KAIDENG	NIHAO	NOISE	ZAIJIAN
GUANDENG	91.7%	4.2%	4.2%	0%	0%
KAIDENG	6.7%	93.3%	0%	0%	0%
NIHAO	0%	0%	100%	0%	0%
NOISE	0%	0%	0%	100%	0%
ZAIJIAN	0%	0%	0%	0%	100%
F1 SCORE	0.94	0.93	0.96	1.00	1.00

Metrics (validation set)



METRIC	VALUE
Area under ROC Curve ?	1.00
Weighted average Precision ?	0.97
Weighted average Recall ?	0.97
Weighted average F1 score ?	0.97



基于Edge Impulse的神经网络设计

■ 模型测试

- 执行分类
- 等待分类完成
- 查看测试精度
- 注意混淆矩阵

Confusion matrix

	GUANDENG	KAI DENG	NIHAO	NOISE	ZAIJIAN	UNCERTAIN
GUANDENG	90.0%	0%	0%	3.1%	0%	0%
KAI DENG	0.1%	90.9%	0%	0%	0%	0%
NIHAO	4%	0%	92%	4%	0%	0%
NOISE	0%	0%	0%	88%	0%	12%
ZAIJIAN	0%	0%	0%	0%	100%	0%
F1 SCORE	0.94	0.95	0.96	0.90	1.00	

The screenshot displays the Edge Impulse web interface. On the left, a sidebar menu includes options like 'Create impulse', 'MFCC', 'Classifier', 'Retrain model', 'Live classification', 'Model testing' (highlighted with a red box and an arrow), 'Perf. calibration', 'Deployment', 'Versioning', 'Documentation', and 'Forums'. The main area is titled 'Test data' and shows a table of test samples. A 'Classify all' button is highlighted with a red box. The 'Model testing output' section shows a JSON report with various metrics. The 'Results' section displays the 'ACCURACY' as 93.44%, which is also highlighted with a red box. Below this, the 'Metrics for Classifier' section shows 'Area under ROC Curve' as 1.00 and 'Weighted average Precision' as 0.95.

SAMPLE NAME	EXPECTED OUTCO...	LENGTH	ACCURACY	RESULT
file_99	noise	1s	100%	1 noise
file_91	noise	1s	100%	1 noise
file_89	noise	1s	100%	1 noise
file_88	noise	1s	0%	1 uncertain
file_85	noise	1s	100%	1 noise
file_8	noise	1s	100%	1 noise
file_79	noise	1s	100%	1 noise
file_72	noise	1s	100%	1 noise

```
{
  "report": {
    "f1": {
      "precision": 1.0,
      "recall": 1.0,
      "f1-score": 1.0,
      "support": 18
    },
    "accuracy": 1.0,
    "macro avg": {
      "precision": 1.0,
      "recall": 1.0,
      "f1-score": 1.0,
      "support": 18
    },
    "weighted avg": {
      "precision": 1.0,
      "recall": 1.0,
      "f1-score": 1.0,
      "support": 18
    },
    "auc": null,
    "loss": 0.0076257881885788,
    "support": 18,
    "weighted_average": {
      "precision": 1.0,
      "recall": 1.0,
      "f1-score": 1.0
    }
  },
  "class_names": [
    "guandeng",
    "kaideng",
    "nihao",
    "noise",
    "zaijian"
  ]
}
```

Generating model testing summary...
Finished generating model testing summary.
Job completed (success)

Results

Model version: Optimized (float32)

ACCURACY 93.44%

Metrics for Classifier

Metric	Value
Area under ROC Curve	1.00
Weighted average Precision	0.95



实验二：神经网络设计

- 基于Edge Impulse，修改设计神经网络模型
 - 分别修改神经网络的滤波器个数、kernel size和内层数各两种情况，记录模型结构，训练精度和混淆矩阵结果
 - 分别增加卷积层、全连接层各两种情况，记录模型结构、训练精度和混淆矩阵结果
- 设计训练一个语音识别性能良好的神经网络模型
 - 模型**训练精度**不低于95%
 - 模型**训练混淆矩阵**中各指令正确率不低于85%
 - 模型**测试精度**不低于90%
 - 模型**测试混淆矩阵**中各指令正确率不低于80%
 - 截图记录网络模型结构、测试精度结果、混淆矩阵



Arduino神经网络语音识别

■ 模型部署

➤ 生成Arduino库。

- Data acquisition
- Experiments
- EON Tuner
- Impulse design
 - Create impulse
 - MFCC
 - Classifier
 - Retrain model
 - Live classification
 - Model testing
 - Perf. calibration
- Deployment

Configure your deployment

You can deploy your impulse to any device. This makes the model run without an internet connection, minimizes latency, and runs with minimal power consumption. [Read more.](#)

Search deployment options



DEFAULT DEPLOYMENT

C++ library

A portable C++ library with no external dependencies, which can be compiled with any modern C++ compiler.

MODEL OPTIMIZATIONS

Model optimizations can increase on-device performance but may reduce accuracy.

Search deployment options



C++ library

A portable C++ library with no external dependencies, which can be compiled with any modern C++ compiler.



Arduino library

An Arduino library with examples that runs on most Arm-based Arduino development boards.



BrainChip MetaTF Model for Akida - AKD1000 and AKD1500

A MetaTF converted model (.fbz) for use with the BrainChip Akida™ runtime.



Cube.MX CMSIS-PACK

A STM32Cube.MX CMSIS-PACK, for fast inferencing on many ST MCUs.



Custom block

A ZIP file containing everything your custom deploy block will receive.



Ethos-U55-128 library (High End Embedded, Shared SRAM)

A C++ library with inferencing for devices with an Ethos-U55-128 NPU, High End Embedded



Arduino神经网络语音识别

■ 模型部署

➤ 生成Arduino库——.ZIP文件

 **EON™ Compiler**
Same accuracy, 39% less RAM, 46% less ROM.

Quantized (int8)
Selected ✓

	MFCC	CLASSIFIER	TOTAL
LATENCY	350 ms.	6 ms.	356 ms.
RAM	15.4K	5.0K	15.4K
FLASH	-	33.8K	-
ACCURACY			-

Unoptimized (float32)
Select

	MFCC	CLASSIFIER	TOTAL
LATENCY	350 ms.	49 ms.	399 ms.
RAM	15.4K	8.6K	15.4K
FLASH	-	32.7K	-
ACCURACY			91.74%

To compare model accuracy, run model testing for all available optimizations.

Run model testing

 **EON™ Compiler**
Same accuracy, 39% less RAM, 46% less ROM.

Quantized (int8)
Selected ✓

	MFCC	CLASSIFIER	TOTAL
LATENCY	350 ms.	6 ms.	356 ms.
RAM	15.4K	5.0K	15.4K
FLASH	-	33.8K	-
ACCURACY			90.91%

Unoptimized (float32)
Select

	MFCC	CLASSIFIER	TOTAL
LATENCY	350 ms.	49 ms.	399 ms.
RAM	15.4K	8.6K	15.4K
FLASH	-	32.7K	-
ACCURACY			91.74%

Estimate for esp32c3 - Change target.

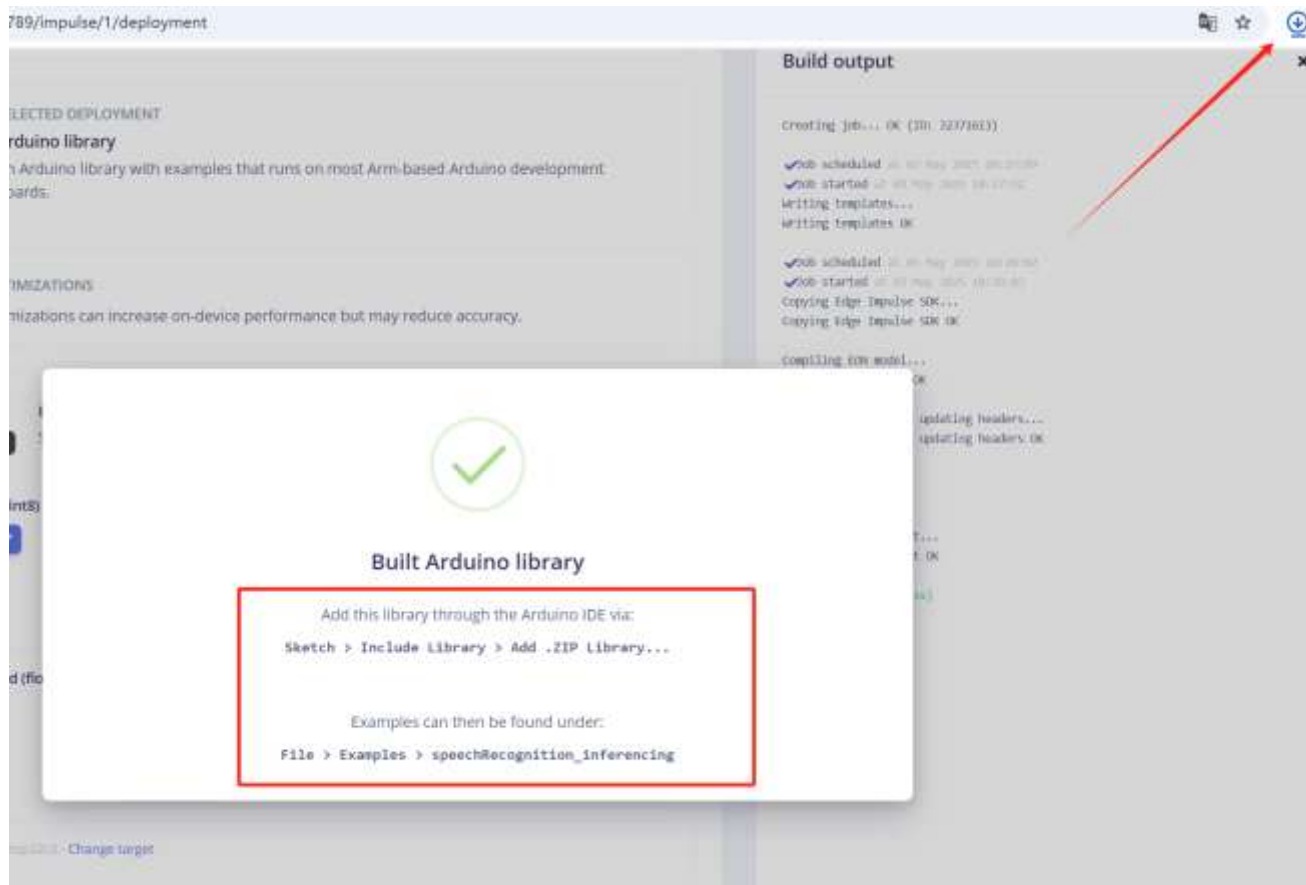
Build



Arduino神经网络语音识别

■ 模型部署

➤ 生成Arduino库——.ZIP文件



ei-speechrecognition-arduino-1.0.1.zip



输出

Library installed



Arduino神经网络语音识别

■ 打开edgeImpulse_esp32.ino

➤ 注意头文件引用是否与自己的库名一致，不一致进行修改

```
/* Includes ----- */  
#include <speechRecognition_inferencing.h>  
  
#include "freertos/FreeRTOS.h"  
#include "freertos/task.h"
```

➤ 验证上传——第一次时间较久（约10分钟以上）





Arduino神经网络语音识别

■ 上传后打开串口监视器

- 说语音指令后，打印采集数据，和各个模块消耗时间
- 各个指令对应概率，概率最高为识别结果，在最后一行打印

输出 串口监视器 x

消息 (按回车将消息发送到"COM45"上的"ESP32C3 Dev Module")

-----请说语音指令-----

DATA 211 77 31 115 33 -25 -105 -87 -138 -113 -78 -52 16 -1 54 141 116 74 10 60 -7 -2 -122 7 8 -29

Predictions (DSP: 1314 ms., Classification: 8 ms., Anomaly: 0 ms.):

guandeng: 0.000000

kaideng: 0.000000

nihao: 0.992187

noise: 0.003906

zaijian: 0.003906

你好

-----请说语音指令-----

```
max_probability = 0;
max_probability_ix = 0;
for (size_t ix = 0; ix < EI_CLASSIFIER_LABEL_COUNT; ix++)
{
    ei_printf("    %s: ", result.classification[ix].label);
    ei_printf_float(result.classification[ix].value);
    ei_printf("\n");
    if (max_probability <= result.classification[ix].value)
    {
        max_probability = result.classification[ix].value;
        max_probability_ix = ix;
    }
}

if (result.classification[max_probability_ix].label == "kaideng")
{
    ei_printf("开灯\n");
    digitalWrite(led, LOW);
}
else if (result.classification[max_probability_ix].label == "guandeng")
{
    ei_printf("关灯\n");
    digitalWrite(led, HIGH);
}
```



实验三：Arduino神经网络语音识别

- 将ESP32-C3实验板和显示控制板上的红色LED灯（数字引脚1）进行连接，实现语音控制LED灯亮灭功能。

指令内容	实现效果
“开灯”	红灯亮起（已有代码）
“关灯”	红灯熄灭（已有代码）
“你好”	红灯以1000ms为周期闪三次后熄灭（自行添加）
“再见”	红灯以400ms为周期闪五次后熄灭（自行添加）



实验四：Arduino神经网络语音识别

- 将ESP32-C3实验板、麦克风及运放实验板和显示控制板上的OLED屏幕进行连接，结合OLED显示屏实验，实现语音指令OLED显示功能

指令内容	实现效果
“开灯”	红灯亮起（已有代码）
“关灯”	红灯熄灭（已有代码）
“你好”	OLED显示屏上显示Hello（自行添加）
“再见”	OLED显示屏上显示Goodbye（自行添加）



同学们开始实验操作！